

Project: Snake Game
Authors: Team 15
Version: 1.0
Date: 06-09-2026
Status: Model Answer

Revision history

Version	Date	Authors	Change summary	Approval
1.0	06-09-2026	Team 15	Initial SRS with diagrams embedded	

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

1. Introduction
2. Overall description
3. External interfaces
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. UML Use-Case Diagram
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document describes the requirements for a single-player Snake game. It is meant to give developers and testers a clear, shared idea of what the game needs to do before any code is written.

1.2 Scope

The game is a classic Snake clone: the player steers a growing snake around a grid, eats food to score points, and loses when the snake hits a wall or itself. It covers a single play session per game (start to game-over) and a locally stored high score. It does not cover multiplayer, online leaderboards, or in-app purchases.

1.3 Audience

Developer(s) building the game, a tester checking it against these requirements, and anyone reviewing the project (e.g. an instructor).

1.4 Definitions

- FPS - Frames Per Second
- UI - User Interface
- FR - Functional Requirement
- NFR - Non-Functional Requirement
- Tick - one step of the game loop (the snake moves one cell per tick)

2. Overall description

2.1 Product perspective

The game is a standalone application (desktop or browser). It has no server component in this version - all game state, including the high score, lives on the player's machine.

2.2 Major product functions

- Show a main menu (Start, High Score, Exit)
- Run the core game loop: move snake, check collisions, update score
- Spawn food and grow the snake when food is eaten
- Detect game-over conditions and show a game-over screen
- Track and persist the high score between sessions
- Let the player pause, resume, and restart

2.3 User roles and characteristics

There is one type of user: the Player. No special skills are assumed beyond knowing how to use arrow keys or WASD.

2.4 Operating environment

Runs on a standard desktop OS (Windows/macOS/Linux) or a modern web browser, with keyboard input. No special hardware is required.

2.5 Constraints

- Must run at a playable, consistent speed on typical consumer hardware
- Grid size and starting speed are fixed for version 1.0 (no user-configurable board size yet)
- No network access required for core gameplay

3. External interface requirements

3.1 User interfaces

A single game screen showing the grid, the snake, the food, and the current score. Separate, simple screens for the main menu and game-over, each with clear text and one or two buttons/prompts.

3.2 Hardware interfaces

- Keyboard: arrow keys or WASD for direction, one key to pause, one key to confirm/restart
- (Optional, if released on mobile) on-screen swipe/touch controls - out of scope for v1.0

3.3 Software interfaces

- Local storage / a small save file for the high score
- Standard OS or browser rendering and input APIs - no external services required

3.4 Communications

None required for gameplay. If an online leaderboard is added in a future version, all requests to that service must use HTTPS/TLS.

4. System features (detailed)

Each requirement below has an ID, a priority, and a way to check it's done. IDs follow SNK-F-###.

4.1 Game initialization and menu

Set up the main menu and starting board state.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-001	The system shall show a main menu with Start, High Score, and Exit options on launch.	Functional	High	Player	Main menu appears with all three options visible and selectable.	-
SNK-F-002	The system shall create a fixed-size grid game board when Start is selected.	Functional	High	Player	Board of the defined width/height is drawn before the first tick.	-
SNK-F-003	The system shall place the snake at the centre of the board with a starting length of 3 segments.	Functional	High	Player	Snake appears centred, 3 segments long, facing a default direction.	-

4.2 Movement and control

Handle the snake's movement and how the player steers it.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-004	The system shall move the snake one cell forward in its current direction on every tick.	Functional	High	Player	Snake advances exactly one cell per tick at the configured tick rate.	-
SNK-F-005	The system shall change the snake's direction when an arrow key or WASD key is pressed, ignoring a reversal directly into itself.	Functional	High	Player	Pressing a valid direction key updates heading within one tick; the reverse direction is ignored.	-
SNK-F-006	The system shall end the game when the snake's head crosses the board boundary.	Functional	High	Player	Hitting any edge of the board triggers game-over immediately.	-

4.3 Food and growth

Handle food spawning and how the snake grows.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-007	The system shall place one food item on a random empty cell whenever no food is present on the board.	Functional	High	Player	A new food item appears on an unoccupied cell within one tick of the previous one being eaten.	-
SNK-F-008	The system shall add one segment to the snake's tail when the snake's head reaches a food cell.	Functional	High	Player	Snake length increases by exactly 1 on each food eaten.	-
SNK-F-009	The system shall increase the player's score by a fixed amount each time food is eaten.	Functional	High	Player	Score increases by the defined point value per food item.	-

4.4 Collision and game over

Handle the conditions that end a game.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-010	The system shall end the game when the snake's head occupies a cell already occupied by its own body.	Functional	High	Player	Self-collision reliably ends the game within one tick.	-
SNK-F-011	The system shall show a game-over screen with the final score after any game-ending collision.	Functional	High	Player	Game-over screen appears and displays the correct final score.	-
SNK-F-012	The system shall offer a restart option on the game-over screen.	Functional	High	Player	Selecting restart starts a fresh game without returning to the main menu.	Depends on SNK-F-002

4.5 Score and persistence

Handle scoring and the saved high score.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-013	The system shall display the current score on screen during gameplay.	Functional	Medium	Player	Score value on screen updates immediately when it changes.	-
SNK-F-014	The system shall save the high score locally and load it again on the next launch.	Functional	High	Player	Closing and reopening the game shows the same high score as before, if beaten.	-
SNK-F-015	The system shall update the stored high score only if the current score is higher than the saved one.	Functional	High	Player	High score never decreases after a lower-scoring game.	-

4.6 Pause and restart

Handle mid-game control.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-016	The system shall pause the game loop when the pause key is pressed during gameplay, and resume it on a second press.	Functional	Medium	Player	Snake stops moving while paused and continues correctly on resume.	-
SNK-F-017	The system shall allow the player to restart directly from the game-over screen.	Functional	Medium	Player	Restart resets score, snake position, and board to initial state.	-

4.7 Difficulty

Handle the game getting harder over time.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-018	The system shall increase the snake's movement speed once the score	Functional	Medium	Player	Tick rate measurably decreases (game speeds up) after crossing each threshold.	-

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	passes each of a set of fixed thresholds.					

5. Non-functional requirements (detailed)

NFRs below are things you can actually measure. IDs follow SNK-NF-####.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
SNK-NF-001	The game loop shall run at a steady 60 FPS on typical consumer hardware, independent of the snake's movement tick rate.	Performance	High	Frame time stays under ~16.7ms on average during a play session. Test: TC-Perf-01
SNK-NF-002	A direction key press shall be reflected in the snake's movement within one game tick.	Performance / Usability	High	Measured input-to-movement delay is at most one tick interval. Test: TC-Perf-02
SNK-NF-003	The game shall not crash during a normal play session, including rapid or repeated key presses.	Reliability	High	No crash observed across a defined set of manual/automated stress inputs. Test: TC-Rel-01
SNK-NF-004	The game shall run on Windows, macOS, and Linux (desktop build) or any current major browser (web build) without code changes.	Portability	Medium	Game launches and plays correctly on each target platform. Test: TC-Port-01
SNK-NF-005	The codebase shall separate input handling, game logic, and rendering into distinct modules.	Maintainability	Medium	Code review confirms the three concerns are not mixed in the same file/class. Test: Code review

5.1 Security

5.1.1 Security objectives

- Keep the saved high score from being silently corrupted or easily edited by hand.
- Make sure any text the player enters (e.g. a name for the high score) can't cause problems for the game or, in the browser version, the page it runs on.

5.1.2 Security requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
SNK-SR-001	The system shall strip or reject HTML/script content from any player name entered for the high score.	Security	High	Entering text with HTML/script tags stores/displays it as plain text only. Test: TC-Sec-01
SNK-SR-002	The system shall limit the player name field to a fixed maximum length.	Security	Medium	Input beyond the limit is truncated or rejected. Test: TC-Sec-02
SNK-SR-003	The system shall store the high score file with a basic integrity check (e.g. checksum) to detect manual tampering.	Security	Medium	A hand-edited save file is detected and the game falls back to a default/reset value. Test: TC-Sec-03

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
SNK-SR-004	The system shall only read from and write to its own save file, not other files on disk.	Security	High	File access during gameplay is limited to the game's own save path. Test: Code review
SNK-SR-005	If an online leaderboard is enabled in a future version, the system shall send all requests over HTTPS/TLS.	Security	Medium	Network capture shows only TLS-encrypted requests when the feature is active. Test: TC-Sec-05 (future)

6. Quality attributes and acceptance tests

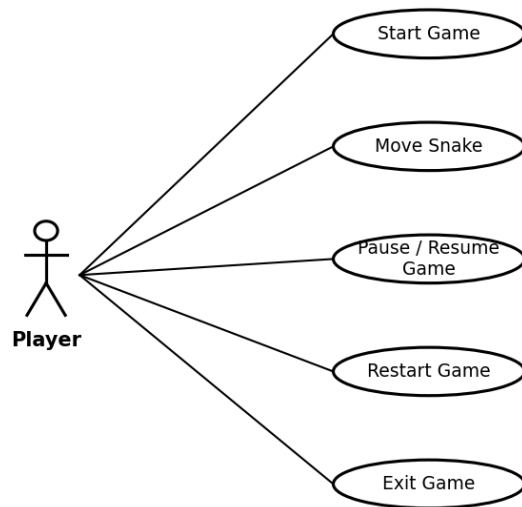
Exit criteria: all High-priority functional requirements are implemented and pass their test, no NFR fails outright, and the RTM (section 8) shows every listed test case passed.

Acceptance test areas: Menu/Start, Movement, Food & Growth, Collision/Game Over, Score & High Score, Pause/Restart, Performance, and Security (name input handling).

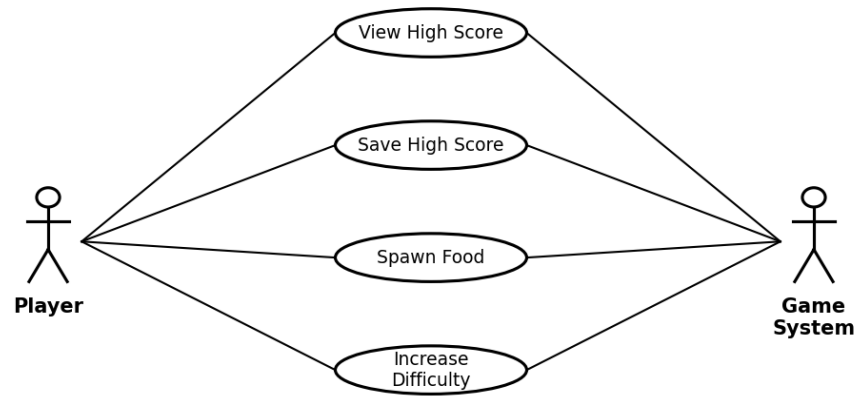
7. UML Use-Case Diagram

Two diagrams below cover the player's direct actions and the score/system-driven behaviour.

Use Case Diagram 1 - Player Interactions



Use Case Diagram 2 - Score & System Behaviour



8. Requirements traceability matrix (RTM)

Req ID	Requirement	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
SNK-F-002	Create board	4.1	BoardModule	TC-Init-01	N	-
SNK-F-005	Change direction	4.2	InputModule	TC-Move-01	N	-
SNK-F-007	Spawn food	4.3	FoodModule	TC-Food-01	N	-
SNK-F-010	Self-collision	4.4	CollisionModule	TC-Coll-01	N	-
SNK-F-014	Save high score	4.5	ScoreModule	TC-Score-01	N	-
SNK-NF-001	60 FPS loop	5	GameLoop	TC-Perf-01	N	-
SNK-SR-001	Sanitize name input	5.1.2	InputModule	TC-Sec-01	N	-

Status key: N = Not tested, P = Pass, A = Fail.